

ROB 550 Armlab Report - AM Station 9

Jimmy Liao, Niva Ranavat, Owen Bigler, Jin-Rou Chang
 {mjliao, nranavat, obigler, jinrou}@umich.edu

Abstract—This report presents the design and implementation of an autonomous robotic system using the Interbotix PincherX 200 (RX200) 5-DOF manipulator and an Intel RealSense L515 RGB-D camera. The project integrates Forward and Inverse Kinematics (FK/IK), Computer Vision (CV), and asynchronous state machine logic to achieve high-precision manipulation. Key developments include a PoE-based kinematic model, a robust camera calibration pipeline using AprilTags, affine/PNP experiments, and homography rectification. Experimental results demonstrate successful autonomous sorting and "Teach and Repeat" functionality, with a quantitative analysis of end-effector positional accuracy and camera-to-world coordinate mapping error across various stack heights. The final system architecture culminates in an autonomous planner capable of executing complex sorting and stacking tasks within the 2026 Armlab competition environment.

I. INTRODUCTION

This report details the development and implementation of an autonomous robotic manipulation system using the Interbotix PincherX 200 (RX200) 5-DOF platform and an Intel RealSense L515 RGB-D camera. The objective of the "Armlab" project is to integrate fundamental principles of robotics—specifically kinematics, computer vision, and state-machine control—to perform complex tasks such as "Teach and Repeat" sequences, precise block stacking, and autonomous sorting.

The system is implemented in Python and interfaces with ROS 2 to manage sensor streaming and motor control. The project progressed through four critical milestones:

- **Low-level Control:** Establishing a GUI-driven interface for manual joint manipulation and waypoint following.
- **Perception and Geometry:** Calibrating camera intrinsics/extrinsics for image-to-world mapping and implementing Forward Kinematics (FK) using a PoE (screw-theory) formulation.
- **Inverse Kinematics and Detection:** Implementing an Inverse Kinematics (IK) solver to enable "click-to-grab" functionality and developing an OpenCV-based pipeline for color-space block detection.
- **Autonomous Planning:** Synthesizing these components into a robust state machine capable of executing tower stacking and block sorting routines and automated workspace organization.

II. METHODOLOGY

A. Camera Calibration & Coordinate Transforms

- **Intrinsic Calibration:** We calibrated the Intel RealSense L515 intrinsics using the ROS `camera_calibration` checkerboard routine. For each run, the calibration estimates (f_x, f_y, c_x, c_y) by minimizing reprojection error

over detected checkerboard corners. We repeated the procedure multiple times (3–5 runs) and aggregated the intrinsics (mean across runs; outliers were observed and discussed in the Results). The intrinsics are used for back-projecting pixel measurements into camera-frame 3D coordinates.

- **Extrinsic Calibration:** We obtained the world-to-camera extrinsic transform using (i) hand-measured alignment and (ii) a PnP-based approach with AprilTag correspondences. We define the transform $H_{cam \leftarrow world}$ such that $\mathbf{p}_{cam} = H_{cam \leftarrow world} \mathbf{p}_{world}$, with translations reported in millimeters.
- **Homography and Grid Protection:** To rectify the board into a bird's-eye view, we compute a planar homography $H_{img \leftarrow board}$ from a set of manually selected point correspondences on the board plane. The homography is used to warp the image and to verify calibration via grid-point projection (reported in Results).
- **Pixel-to-World Mapping Using Depth:** Given a pixel measurement (u, v) and depth d , we first back-project into the camera frame using the pinhole model:

$$\mathbf{p}_{cam} = \begin{bmatrix} x_c \\ y_c \\ z_c \\ 1 \end{bmatrix} = \begin{bmatrix} (u - c_x) d / f_x \\ (v - c_y) d / f_y \\ d \\ 1 \end{bmatrix}.$$

We then map to the world frame using the extrinsic transform:

$$\mathbf{p}_{world} = H_{world \leftarrow cam} \mathbf{p}_{cam}.$$

To convert image points $[[u, v, d]]$ to world coordinates $[[x, y, z]]$ we used the following algorithm:

1) Undo Homography Transformation

Revert the homography transformation to recover the original image points.

$$\begin{bmatrix} u_{orig} \\ v_{orig} \\ 1 \end{bmatrix} = H_{hom}^{-1} \begin{bmatrix} u_{warped} \\ v_{warped} \\ 1 \end{bmatrix}$$

2) Image to Camera Frame:

Points in the image $[[u, v, 1]]^T$ are scaled by depth d and multiplied by the inverse intrinsic matrix K^{-1} :

$$\begin{bmatrix} x_c \\ y_c \\ z_c \end{bmatrix} = d \cdot K^{-1} \begin{bmatrix} u \\ v \\ 1 \end{bmatrix}$$

3) Camera to World Frame:

Using the extrinsic parameters and the inverse of the homogeneous

matrix H :

$$\begin{bmatrix} x_w \\ y_w \\ z_w \\ 1 \end{bmatrix} = H_{cam \leftarrow world}^{-1} \begin{bmatrix} x_c \\ y_c \\ z_c \\ 1 \end{bmatrix}$$

B. Computer Vision Pipeline

- **Block Detection:** Unlike traditional color-first approaches, we adopt a depth-first, two-layer geometric-slicing strategy:

- 1) **Footprint Extraction:** We perform background subtraction using a pre-saved base depth map to isolate physical objects ($\Delta h > 5$ mm). After filtering out the robot arm and masking boundary blind zones to prevent edge flickering, morphological operations (Open/Close) are applied to consolidate broken segments into primary contours.
- 2) **Top-Face Slicing (Layer 2 Analysis):** To eliminate perspective distortion where the camera sees the vertical side-walls of the blocks, the algorithm computes the 95th-percentile height within each block's footprint. It then applies a 3 mm vertical threshold window below this peak to isolate the block's flat top surface.
- 3) **Geometric Distractor Filtering:** We compute the convex hull of the isolated top face. By evaluating the aspect ratio (target ≈ 1.0) and rectangularity fill ratio (target > 0.8), the system rejects non-target geometric distractors such as arches and triangles.
- 4) **Color Labeling:** After the top-face centroid is determined, we classify color using an HSV-based classifier (details and thresholds in the **Color Classification** paragraph below).

- **Color Classification:** The HSV hue thresholds were calibrated on the physical blocks and are summarized in Table I. During inference, we sample a 5×5 RGB patch centered at the detected top-face centroid, convert it to HSV, and assign a color label by thresholding the hue into six ranges (Red, Orange, Yellow, Green, Blue, Violet). The selected thresholds were validated on the real blocks.
- **State Machine:** The system is implemented as a state machine with event-driven logic where GUI interactions or detector outputs trigger target selection, IK solver, and gripper states.

TABLE I
CALIBRATED HSV HUE RANGES FOR BLOCK CLASSIFICATION

Color	Hue Range (H)	Reference Center
Red	$H \geq 155$ or $H \leq 1$	176 / 0
Orange	$2 \leq H \leq 13$	4
Yellow	$14 \leq H \leq 45$	23
Green	$46 \leq H \leq 88$	73
Blue	$89 \leq H \leq 108$	103
Violet	$109 \leq H \leq 154$	124

TABLE II
SPATIAL SCREW AXES PARAMETERS FOR RX200

Joint (i)	ω_{ix}	ω_{iy}	ω_{iz}	v_{ix}	v_{iy}	v_{iz}
1	0	0	1	0	0	0
2	0	1	0	-0.10457	0	0
3	0	1	0	-0.30457	0	0.05
4	0	1	0	-0.30457	0	0.25
5	1	0	0	0	0.30457	0

C. Kinematics & Motion Planning

1) **Forward Kinematics (FK):** Figure 1 shows the FK schematic and frame conventions used in our PoE model.

For the implementation of Forward Kinematics (FK), Interbotix provides the home configuration matrix M and the screw list ($\mathcal{S}$) (Table II) based on screw theory. Therefore, we discarded the traditional Denavit-Hartenberg (D-H) parameter method and directly adopted the Product of Exponentials (PoE) formula for our calculations:

$$T(\theta) = e^{[\mathcal{S}_1]\theta_1} e^{[\mathcal{S}_2]\theta_2} \dots e^{[\mathcal{S}_n]\theta_n} M,$$

where $\mathcal{S}_i = [\omega_i, v_i]^T$ are spatial screw axes and M is the home configuration.

$$M = \begin{bmatrix} 1 & 0 & 0 & 0.408575 \\ 0 & 1 & 0 & 0.0 \\ 0 & 0 & 1 & 0.30457 \\ 0 & 0 & 0 & 1 \end{bmatrix}.$$

However, we observed that the Base coordinate frame defined by the manufacturer's M matrix differs in orientation from the World coordinate frame defined for our actual workspace. While their origins coincide, there is a relative rotation of 90 degrees around the Z-axis. To maintain the generality and independence of the core kinematics module (kinematics.py), we did not directly modify the original M matrix. Instead, in the `get_ee_pos` function within `rxarm.py`, after calling the FK to compute the end-effector pose, we applied an additional coordinate transformation using homogeneous transformation matrices. The transformation matrix T_{world_base} from the Base frame to the World frame is defined as follows:

$$\theta = \pi/2$$

$$T_{world_base} = \begin{bmatrix} \cos(\theta) & -\sin(\theta) & 0 & 0 \\ \sin(\theta) & \cos(\theta) & 0 & 0 \\ 0 & 0 & 1 & 0 \\ 0 & 0 & 0 & 1 \end{bmatrix}$$

In addition to the base transformation, we also performed a corresponding pose correction for the Tool frame at the gripper's end. To accommodate the system's subsequent use of ZYZ Euler Angles for pose control and to handle the specific phase relationships inherent to Euler angles (e.g., we define the end-effector orientations facing each axis as: X+ is $[0^\circ, 90^\circ, 90^\circ]$, Y+ is $[90^\circ, 90^\circ, 90^\circ]$, and Z- is $[-90^\circ, 180^\circ, -90^\circ]$), we introduced a tool correction matrix $T_{tool_correction}$:

$$T_{tool_correction} = \begin{bmatrix} 0 & 0 & 1 & 0 \\ -1 & 0 & 0 & 0 \\ 0 & -1 & 0 & 0 \\ 0 & 0 & 0 & 1 \end{bmatrix}$$

By combining these two corrections, the final, complete pose matrix of the robotic arm's end-effector in the world coordinate system, ${}^{ee}T_{world}$, can be elegantly obtained through pure matrix multiplication:

$${}^{ee}T_{world} = T_{world_base} {}^{ee}T_{base} T_{tool_correction}.$$

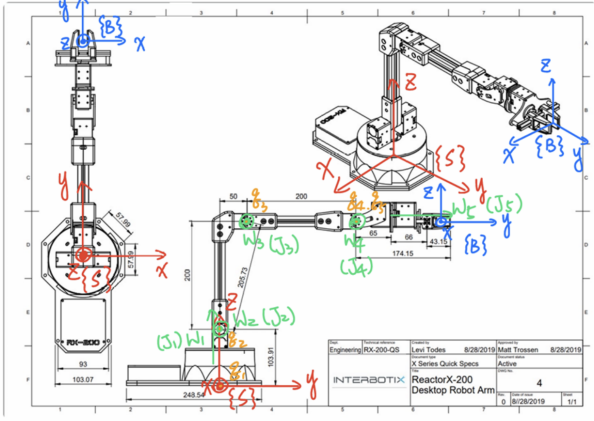


Fig. 1. Forward Kinematics Schematic Drawing for our Armlab project.

2) Inverse Kinematics (IK): We solve IK using a geometric decoupling approach with two grasping postures (vertical vs. degenerate horizontal) and select the elbow-up/down branch using a seed-distance cost. To better diagnose the root causes of positioning errors, we adopted a geometric and algebraic analytical approach to calculate the Inverse Kinematics (IK). First, we decouple the arm's five degrees of freedom (DOF) to reduce the dimensionality of the problem. When a target position in space is specified, its XY coordinates directly define the angle of the J_1 axis (waist). However, it is important to note that the XY axes of the robot's default base frame (which can be observed in the Product of Exponentials M matrix) are inverted relative to our defined workspace XY coordinates. Therefore, J_1 is calculated as follows:

$$x_{base} = y_w$$

$$y_{base} = -x_w$$

$$q_1 = \text{atan2}(y_{base}, x_{base})$$

Next, based on the reachable workspace, we define two grasping postures for the end-effector: a vertical grasp and a horizontal grasp. For a vertical grasp, we want the gripper to remain horizontal while grasping the block; thus, J_5 acts as a compensation angle for J_1 , determined by the block's orientation (detected via the block detection algorithm) and the current J_1 angle. If the target position is further away, the system switches to a horizontal grasp, which is considered a

degenerate pose. In this degenerate state, the J_5 angle is fixed to keep the grasping direction parallel to the table surface.

Because we constrained the arm to these two distinct grasping postures, the J_4 angle naturally serves as a compensation angle for J_2 and J_3 to lock in the desired end-effector orientation.

Through this decoupling strategy, the computation for the five DOF is simplified down to just two remaining variables: J_2 (shoulder) and J_3 (elbow). Depending on the target grasping posture, we apply an upward or depth-wise offset to the wrist position. These two joints and the target point will then lie on a single 2D plane perpendicular to the workspace table. By converting the XYZ coordinates of the target point into a height Z and a planar depth $D_{xy} = \sqrt{X_w^2 + Y_w^2}$.

The 3D spatial problem is successfully reduced to two unknowns and two equations. The schematics are shown in Figure 2.

$$P_{wD} = -200 \sin \theta_2 + 50 \cos \theta_2 + 200 \cos(\theta_2 + \theta_3)$$

$$P_{wH} = 200 \cos \theta_2 + 50 \sin \theta_2 + 200 \sin(\theta_2 + \theta_3)$$

Having simplified the problem geometrically, we directly compute the exact closed-form solution using algebraic operations. We restructure the kinematic equations into the standard form of $A \cos \theta + B \sin \theta = C$.

Using trigonometric identities and the arccos function, we can solve for the two possible solutions of θ_2 (representing the elbow-up and elbow-down configurations) in a single step, and then substitute θ_2 back to solve for θ_3 . Finally, the algorithm introduces a distance cost function between these two valid solutions and the current joint positions (the seed values). It automatically selects the optimal solution that yields the shortest and smoothest movement trajectory.

IK Implementation:

- **Posture selection:** compute $r^2 = x_w^2 + y_w^2$ and z_{curve} (Eq. 1); choose VERTICAL_DOWN if $z_w \leq z_{curve}$, else HORIZONTAL_OUT.
- **Base rotation and planar target:** compute $(q_1, P3D, P3H)$ from Eqs. 2–3.
- **Closed-form (θ_2, θ_3) :** compute (M, N, K) and solve for θ_2 using Eqs. 4–7; recover θ_3 and select the seed-closest branch (Eq. 6).
- **Mapping:** map to joint angles using Eq. 7 and clamp to limits.

$$z_{curve} = -1.4263 \times 10^{-8} (r^2)^2 + 5.9813 \times 10^{-4} r^2 + 270.26 \quad (1)$$

$$x_{base} = y_w, \quad y_{base} = -x_w, \quad q_1 = \text{atan2}(y_{base}, x_{base}) \quad (2)$$

$$P3D = D_{xy} - L_{eff} \cos \alpha, \quad P3H = (z_w + L_{eff} \sin \alpha) - h_{J2}. \quad (3)$$

$$\begin{aligned} M &= 400P3D - 100P3H, \\ N &= -100P3D - 400P3H, \\ K &= P3D^2 + P3H^2 + 2500. \end{aligned} \quad (4)$$

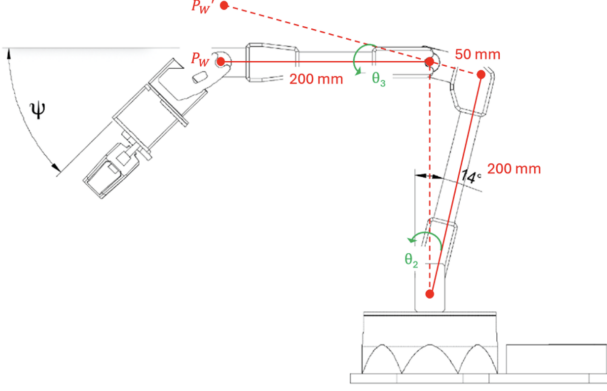


Fig. 2. Inverse Kinematics schematic drawing for our Armlab project.

$$\begin{aligned} \theta_2^{(1,2)} &= \phi \pm \arccos\left(-\frac{K}{R}\right), \\ R &= \sqrt{M^2 + N^2}, \\ \phi &= \text{atan2}(M, N). \end{aligned} \quad (5)$$

$$J = (\theta_2 - \theta_{2,seed})^2 + (\theta_3 - \theta_{3,seed})^2. \quad (6)$$

$$q_2 = -\theta_2, \quad q_3 = -\theta_3, \quad q_4 = \alpha - (q_2 + q_3). \quad (7)$$

Code Reference: `IK_basic()` is implemented in `kinematics.py` (lines 144–220) and calls the closed-form planar solver `solve_theta2_theta3_exact()` (lines 103–143). The pseudocode for the planar solver is summarized in Algorithm 1.

3) Safety and Feasibility Constraints: To ensure operational stability and prevent physical damage, we implemented several safety and logic layers. Before invoking the Inverse Kinematics (IK) function, the system calculates both the target point and the required pre-grasp hovering point (generated by applying an upward Z-axis offset to the target). Both sets of coordinates are then evaluated against this fitted curve. If either point’s altitude exceeds the height boundary z defined by the equation, the system forces the gripper to maintain a horizontal grasping posture. This logic is implemented to prevent any undesired posture transitions from occurring while the arm is executing its vertical descent from the hovering point to the target point:

- **Gesture selection (workspace-dependent):** The solver switches between two end-effector approach gestures based on a workspace boundary function (“Z-curve”): `VERTICAL_DOWN` for top-down grasps and `HORIZONTAL_OUT` for extended reach / forward approach and reaching further distances.
- **Joint-limit clamping:** All IK solutions are passed through `clamp_with_limits` to enforce RX200 joint bounds. The joint bounds used in the clamping and IK reachability logic are summarized in Table III.

Algorithm 1 Closed-form solver for (θ_2, θ_3) (`kinematics.py`, lines 103–143)

```

1: procedure SOLVETHETA2THETA3EXACT( $P3D, P3H$ )
2:   Input:  $P3D, P3H, \theta_{2,seed}, \theta_{3,seed}$ 
3:    $M \leftarrow 400 \cdot P3D - 100 \cdot P3H$ 
4:    $N \leftarrow -100 \cdot P3D - 400 \cdot P3H$ 
5:    $K \leftarrow P3D^2 + P3H^2 + 2500$ 
6:    $R \leftarrow \sqrt{M^2 + N^2}$ 
7:   if  $R = 0$  or  $|\frac{-K}{R}| > 1$  then
8:     return  $(\theta_{2,seed}, \theta_{3,seed})$  ▷ Unreachable
9:    $\phi \leftarrow \text{atan2}(M, N)$ 
10:   $a \leftarrow \arccos(\text{clip}(-\frac{K}{R}, -1, 1))$ 
11:   $\theta_2^{(1)} \leftarrow \phi + a, \quad \theta_2^{(2)} \leftarrow \phi - a$ 
12:   $\mathcal{C} \leftarrow \emptyset$  ▷ Candidate solutions
13:  for  $\theta_2 \in \{\theta_2^{(1)}, \theta_2^{(2)}\}$  do
14:     $X \leftarrow P3D + 200 \sin \theta_2 - 50 \cos \theta_2$ 
15:     $Y \leftarrow P3H - 200 \cos \theta_2 - 50 \sin \theta_2$ 
16:     $\theta_{23} \leftarrow \text{atan2}(Y, X)$ 
17:     $\theta_3 \leftarrow \theta_{23} - \theta_2$ 
18:     $\theta_2 \leftarrow \text{atan2}(\sin \theta_2, \cos \theta_2)$  ▷ Normalize
19:     $\theta_3 \leftarrow \text{atan2}(\sin \theta_3, \cos \theta_3)$ 
20:     $\mathcal{C} \leftarrow \mathcal{C} \cup \{(\theta_2, \theta_3)\}$ 
21:   $(\theta_2^*, \theta_3^*) \leftarrow \arg \min_{(\theta_2, \theta_3) \in \mathcal{C}} [(\theta_2 - \theta_{2,seed})^2 + (\theta_3 - \theta_{3,seed})^2]$ 
22:  return  $(\theta_2^*, \theta_3^*)$ 

```

TABLE III
MANIPULATOR JOINT LIMITS

Joint	Lower Limit (°)	Upper Limit (°)
J_1	-180	180
J_2	-108	113
J_3	-108	93
J_4	-100	123
J_5	-180	180

- **Reachability checks:** Targets outside the arm’s reachable workspace are rejected to avoid invalid configurations (e.g., NaNs) and unstable poses.

4) Motion Planning & Control: We execute pick-and-place using a waypoint-based planner designed to reduce collisions and prevent orientation flips:

- **Approach strategy (hover → descend):** For each pick/place, the arm moves to a hover pose above the target and then descends vertically to the grasp/release pose.
- **Gesture-locked waypoint calculation:** We first solve IK at the target to obtain (q_{target}, G_{target}) , then force the hover waypoint to use the same gesture to avoid re-orientation during vertical motion:

$$q_{hover} = f_{IK}(P_{hover}, \psi, \text{force_gesture} = G_{target}).$$
- **Dynamic motion profile:** End-effector velocity is regulated using a distance-based temporal scaling model. The moving time T_m is calculated relative to the Euclidean

distance between the current pose P_c and target pose P_t :

$$T_m = \max \left(2.0, \frac{\|P_t - P_c\|_2}{200} \right) \quad (8)$$

The acceleration time is set to $0.2 \times T_m$ to provide a smooth trapezoidal velocity profile, reducing motor stress and mechanical oscillation during high-speed transits.

- **Obstacle Avoidance Sequence:** The execution follows a 9-step state sequence designed to clear the workspace. A critical safety feature is the neutral retreat (Step 5), which returns the arm to the initial pose $q_{init} = [0, -0.87, 0.52, -0.87, 0]$ between the pick and place phases to avoid mid-workspace collisions. The motion sequence is listed in Table IV and summarized procedurally in Algorithm 2.

TABLE IV
PICK-AND-PLACE EXECUTION SEQUENCE

Phase	Step	Operational Action
Pick	1–2	Approach Hover $\rightarrow$ Descend to Target
	3–4	Actuate Gripper (Grasp) $\rightarrow$ Vertical Lift
Transit	5	Neutral Retreat to q_{init}
Place	6–7	Cross to Place Hover $\rightarrow$ Descend to Target
	8–9	Actuate Gripper (Release) $\rightarrow$ Home Pose

Algorithm 2 Dynamic Pick-and-Place Execution

```

1: procedure EXECUTETASK( $P_{pick}, P_{place}, \psi$ )
2:    $q_{pick}, G \leftarrow IK(P_{pick}, \psi)$   $\triangleright$  Infer Gesture
3:    $q_{hover} \leftarrow IK(P_{hover}, \psi, \text{force} = G)$ 
4:   if  $q$  is invalid then return Error
5:   Move to  $q_{hover}$  with  $T_m$ 
6:   Grasp and Retreat to  $q_{init}$ 
7:   Repeat for Place sequence

```

III. RESULTS

A. Checkpoint 1

1) **Teach and Repeat:** For the teach-and-repeat cycle task, we first demonstrated a pick-and-place trajectory and recorded waypoints at 0.5 s intervals with a per-segment move time of 0.5 s. The robot then replayed the recorded trajectory repeatedly. The cycle task swaps blocks between $(x, y) = (-100, 225)$ and $(100, 225)$ (mm) via an intermediate waypoint at $(250, 75)$ in the world frame. One cycle follows the sequence: pick at the source location $\rightarrow$ transit via $(250, 75)$ $\rightarrow$ place at the destination location $\rightarrow$ return via $(250, 75)$ for the next swap. We achieved 5 successful cycles; after the fifth repetition, accumulated variability in block placement and pickup caused a gradual pose drift, eventually leading to missed grasps at the originally taught pickup location. Joint trajectories for one representative cycle are shown in Figure 3.



Fig. 3. The Joint Angles Over One Cycle of Teach and Repeat vs. Time

2) *Intrinsic Calibration:*

- Factory-calibrated intrinsic matrix:

$$\begin{bmatrix} 731.0546875 & 0 & 515.859375 \\ 0 & 732.0859375 & 402.69140625 \\ 0 & 0 & 1 \end{bmatrix} \quad (9)$$

- Checkerboard-calibrated intrinsic matrix:

$$\begin{bmatrix} 852.56113 & 0 & 612.92890 \\ 0 & 855.3165 & 345.13327 \\ 0 & 0 & 1 \end{bmatrix} \quad (10)$$

Compared to the factory calibration, the checkerboard calibration produced larger focal lengths and a shifted principal point, which contributed to noticeable back-projection errors in world coordinates. We took the average of 4 different calibration runs. During the project, we trusted the factory matrix more because we were unable to get the correct world coordinates with this intrinsic matrix. We determined the error by using the matrix and the extrinsic to determine the world coordinates. With this intrinsic matrix, a world coordinate of $(-300, -75, 0)$ resulted in roughly $(-270, -5)$.

During checkpoint 1, our extrinsic matrix was

$$H = \begin{bmatrix} 1.000 & 0.000 & 0.000 & 0.000 \\ 0.000 & -0.988 & -0.156 & 270.000 \\ 0.000 & 0.156 & -0.988 & 900.000 \\ 0.000 & 0.000 & 0.000 & 1.000 \end{bmatrix} \quad (11)$$

The rotation angle we used was 171 degrees, and a translation of $(270, 900)$ from the world frame to the camera frame.

B. Checkpoint 2

The new extrinsic matrix using the affine transformation is

$$T_{cam \leftarrow world} = \begin{bmatrix} 0.9999 & -0.0016 & -0.0153 & 18.99 \\ 0.0002 & -0.9933 & 0.1151 & 170.0 \\ -0.0155 & -0.1151 & -0.9932 & 1016.0 \\ 0 & 0 & 0 & 1 \end{bmatrix} \quad (12)$$

Compared to the hand-measured extrinsic from Checkpoint 1, the affine-estimated extrinsic in Checkpoint 2 yields noticeably improved alignment near the board center, but residual error increases toward the periphery. This suggests the nominal transform is sufficient for coarse localization,

while accurate corner localization likely requires distortion-aware calibration or a higher-fidelity pose model. Our calibration pipeline began with the intrinsic matrix derived in the previous stage. To determine the extrinsic parameters, we applied an affine transformation based on the image and world coordinates of three AprilTags. Using the combined intrinsic and extrinsic matrices, we performed a back-projection from image space to world coordinates. While the resulting projection remains highly accurate near the center of the grid, we observed increasing reprojection errors toward the board's periphery, likely due to lens distortion or the limitations of the affine model.

To verify the integrity of our camera calibration, we used global grid projection and point-to-point coordinate verification. First, we generated a synthetic meshgrid based on the physical dimensions of the board and projected it onto the image using our homography and intrinsic matrices, shown in Figure 4. By overlaying this digital grid onto the raw image, we could visually inspect the alignment. A small residual misalignment was observed between the projected grid points and the corresponding board features, but the overlay showed close agreement overall. In addition, we used a custom GUI to select pixel coordinates of known landmarks, transformed them into world coordinates, and compared the results against ground-truth board measurements to estimate the systematic offset of our calibration.

During the verification process, we attempted to improve the extrinsic calibration by accounting for the 3D geometry of the scene. We substituted one of the planar AprilTags with a tag located at a different height (Z-axis elevation) to see if providing the algorithm with 3D spatial depth would refine the projection. However, this adjustment did not result in a measurable increase in accuracy and, in some areas, introduced more instability into the transformation. We also evaluated a Perspective- n -Point (PnP) pose-estimation approach. In our setup, the affine model produced more stable and consistent image-to-world mappings on the planar grid board, whereas PnP exhibited larger variability in the projected alignment. Therefore, we adopted the affine transform as the final extrinsic model for the planar board calibration.

The homography matrix was

$$H_{hom} = \begin{bmatrix} 1.240 & -0.077 & -193.346 \\ 0.007 & 1.079 & -71.574 \\ 0.000 & -0.000 & 1.000 \end{bmatrix} \quad (13)$$

For a planar homography,

$$\begin{bmatrix} u \\ v \\ 1 \end{bmatrix} \sim H \begin{bmatrix} x \\ y \\ 1 \end{bmatrix}, \quad H = \begin{bmatrix} h_{11} & h_{12} & h_{13} \\ h_{21} & h_{22} & h_{23} \\ h_{31} & h_{32} & h_{33} \end{bmatrix},$$

the 2×2 block $\begin{bmatrix} h_{11} & h_{12} \\ h_{21} & h_{22} \end{bmatrix}$ captures the dominant in-plane linear mapping (rotation, scaling, and shear), while (h_{13}, h_{23})

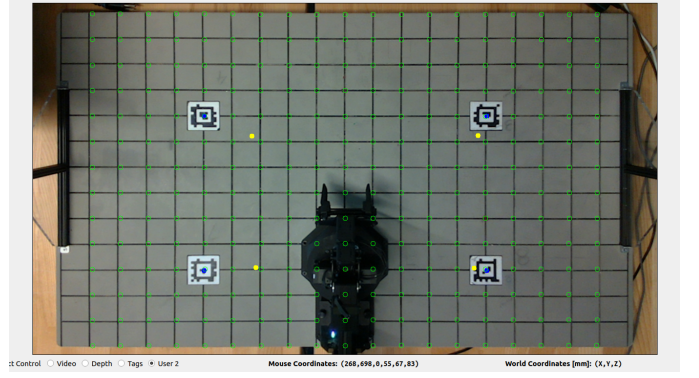


Fig. 4. World coordinate grid points projected on the RGB image using our camera calibration results.

translates points in the image. The bottom row (h_{31}, h_{32}) introduces projective (perspective) effects through the denominator $w = h_{31}x + h_{32}y + h_{33}$; when $h_{31} \approx h_{32} \approx 0$, the mapping is close to affine. In our matrix, h_{31} and h_{32} are near zero, indicating the mapping is close to affine for the planar board under our camera setup.

To visualize the geometric mapping, we first computed the homography matrix using the four AprilTag markers. We then generated a synthetic meshgrid representing the physical dimensions of the grid board, spanning $x \in [-450, 450]$ and $y \in [-175, 625]$. By sampling this world-space grid at 50-unit intervals, we projected each coordinate point into the image plane. This allowed us to overlay a calibrated reference grid directly onto the source image to verify projection accuracy.

1) Forward Kinematics: Details of the FK formulation and frame/tool corrections are described in section II-C1; here, we report the physical verification results. To rigorously verify that our forward and inverse kinematics successfully commanded the arm tooltip to the expected workspace coordinates, we conducted physical measurements using a static, pre-calibrated grid board. We tested the system against the specific target coordinates required in the project checkpoints: $(0, 175, 100)$, $(-300, -75, 25)$, $(300, -75, 50)$, and $(300, 325, 10)$ (in mm). Once the arm stabilized at each target configuration, we visually and physically inspected the position of the closed gripper's center tip relative to the expected spatial locations. Regarding the estimation of positioning error, it is important to note that pinpointing the absolute physical center of the parallel gripper with the naked eye for precise manual measurement is inherently challenging. Furthermore, the Dynamixel servo motors in each joint exhibit a certain degree of unavoidable mechanical backlash. Despite these measurement constraints and hardware limitations, based on the arm's actual reached positions at the aforementioned checkpoint targets, we roughly estimate the positional error to be less than 3 mm across all axes. Given the gripper jaw span and measurement uncertainty, a sub-3 mm positional error is reasonable and supports the validity of our kinematic model. In practice, this accuracy enabled reliable autonomous pick-and-place for our experimental tasks.

C. Checkpoint 3

1) **Block Detection and Color Classification:** HSV calibration thresholds are reported in section II-B ; here we focus on detector performance. After block candidates are segmented, we apply OpenCV contour detection to estimate the image-plane centroid (u, v) and orientation angle θ for each detection. We then associate the corresponding depth measurement from the L515 to recover stack height and back project the 3D position in the world frame (Figure 7). The spatial distribution of localization error across the board is visualized as a heatmap in Figure 5.

Overall, the detector reliably distinguished blocks by both color and size. Our geometric filtering rejected several non-target distractors (e.g., arches and triangles); however, cylindrical objects remained the primary failure mode due to rotational symmetry and limited discriminative contour features.

2) **Inverse Kinematics:** To validate the theoretical accuracy of our kinematic models, we performed a software-level closed-loop consistency check. We utilized fundamentally different mathematical frameworks for our kinematics: the Product of Exponentials (PoE) based on screw theory for Forward Kinematics (FK), and a planar geometric/algebraic approach for Inverse Kinematics (IK). By feeding the joint angles $[\theta_1, \dots, \theta_5]$ calculated by our geometric IK solver back into the PoE FK algorithm, we reconstructed the end-effector's target pose. Across all software test cases, the computational Euclidean error between the original target and the reconstructed pose was 0.0 mm. Because the FK and IK were derived through independent mathematical pathways, this zero-error loop closure (within numerical precision) strongly supports the algebraic correctness and internal consistency of our FK/IK implementations prior to hardware deployment.

To define the operational range for the degenerate posture, we empirically measured the maximum altitude at which the arm could sustain a vertical grasping posture across various depths. Using MATLAB, we then fitted these measurements into a surface profile, yielding the following boundary equation:

$$z = -1.4263 \times 10^{-8}(r^2)^2 + 5.9813 \times 10^{-4}(r^2) + 270.26$$

where $r^2 = x^2 + y^2$ represents the squared radial distance from the robot's base. Figure 6 shows the measured samples

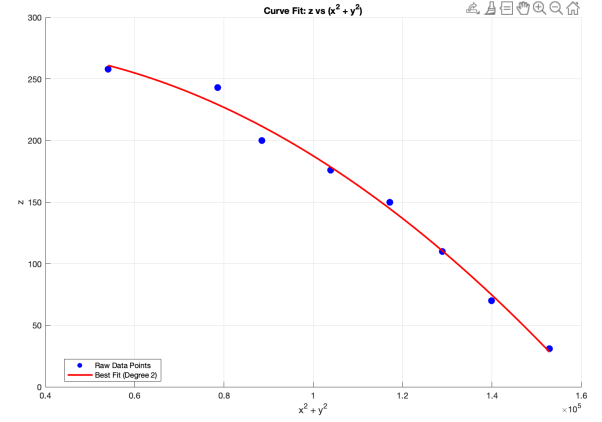


Fig. 6. Operational boundary curve for vertical grasping postures. The plot illustrates the empirically measured maximum achievable height (z) for a vertical grasp against the squared radial distance ($r^2 = x^2 + y^2$) from the robot's base. The blue dots represent the raw physical measurement data, while the solid red line depicts the 2nd-degree polynomial best fit. This fitted boundary is utilized by the IK solver to dynamically trigger the transition into a horizontal (degenerate) grasping posture to ensure safe execution.

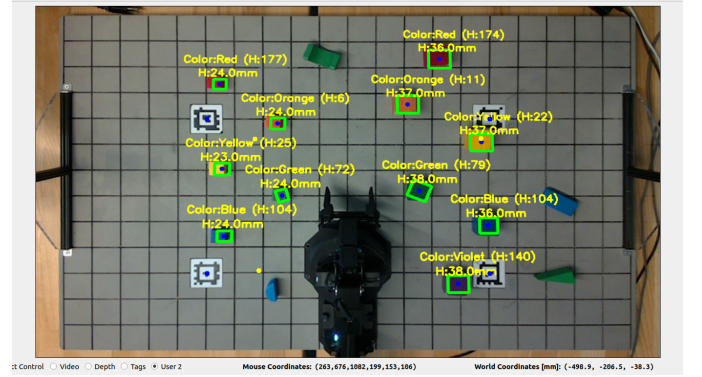


Fig. 7. Result of the block detection algorithm. Each detection includes the color, the HSV value, and the height of the block that has been detected. Most distractors were rejected using this algorithm.

and the fitted boundary. The fitted boundary was used as a posture-selection constraint in the IK controller (described in section II-C3).

D. Checkpoint 4

The pick-and-place state sequence and execution procedure are defined in Section II-C4 (Table IV, Algorithm 2). During the competition, our system successfully completed Level 1, demonstrating reliable performance in basic block manipulation. However, as we progressed to Level 2 and 3, performance was hampered by detection inaccuracies and physical interference. Our color detection remained robust, and we successfully implemented a "trash pile" for sorting. Most distractor shapes were rejected; the dominant remaining ambiguity was cylindrical objects, whose near-circular top-down contours occasionally resembled valid targets. Furthermore, we encountered significant calibration drift at the edges of the grid; while projections were accurate near the center, corner locations exhibited noticeable offsets. This caused the

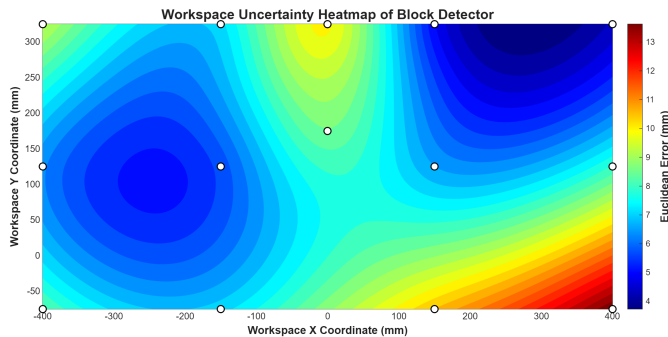


Fig. 5. Positional error heatmap for the computer vision pipeline. Color variations indicate the magnitude of Euclidean error between detected block centroids and actual ground-truth world coordinates across the grid board.

gripper to contact block corners rather than centroids, leading to failed grasps. Additionally, our motion planning did not explicitly handle degenerate configurations or occlusions, so targets blocked by other objects were difficult to retrieve. In Event 3, we stacked a maximum of 10 blocks using a sequential one-by-one stacking strategy.

To improve future performance, several key enhancements are necessary. First, the detection pipeline requires a stronger distractor rejection (e.g., additional depth/geometric cues). Second, the Inverse Kinematics (IK) and trajectory planning should be upgraded to include obstacle avoidance and the ability to maneuver around non-target blocks. Finally, refining the coordinate transformation to account for lens distortion at the board’s periphery would resolve the grasping offsets seen in the corners. Integrating a more dynamic motion planning algorithm would allow the system to handle complex, cluttered environments where simple linear movements are insufficient. For stacking, an improvement could be to make small sub-stacks and then merge them so that we could stack more, which requires a reliable estimation of the stack base pose rather than only the top surface. Overall, our system demonstrated stable end-to-end performance in the competition, successfully completing Level 1 and achieving a maximum stack of 10 blocks in Event 3.

IV. DISCUSSION

A major hurdle in our performance was the significant environmental and geometric degradation in the $-x, -y$ quadrant of the grid. This specific section of the board suffered from inconsistent lighting and severe peripheral lens distortion, which heavily skewed our Computer Vision (CV) reliability. These distortions made it difficult for the camera to maintain a consistent “depth” perception, causing our world-coordinate projections to drift. Additionally, our contour-based filtering handled most non-target shapes reliably. The dominant limitation was cylinder detection, where rotational symmetry and limited discriminative contour features reduced separability; this could be improved with richer geometric cues or a learned classifier. A more robust approach for future iterations would be to move away from purely 2D image processing and instead utilize depth sensor data to build a more accurate mask, which would allow the system to filter out distractors based on 3D volume rather than inconsistent 2D shapes.

Regarding the mechanical execution, we initially attempted to define the workspace in a broad spherical space, but this didn’t align perfectly with the robot’s actual physical constraints. We found that the Inverse Kinematics (IK) solver performed much more reliably once we refined the coordinate mapping to stay strictly within the robot’s natural range of motion. For future iterations, keeping the workspace limits tightly coupled to the robot’s physical reach, rather than a theoretical coordinate bubble, would prevent the arm from attempting unreachable “degenerate” poses and improve the overall success rate of the grasps.

V. CONCLUSION

In summary, our system established a baseline see-and-pick pipeline and performed reliably on Level 1 tasks. We mapped

the grid board using homography and completed basic sorting via HSV-based color classification with a designated “trash pile” strategy. However, higher competition levels exposed reduced robustness to distractors in our vision filtering and revealed calibration drift near the board periphery, which degraded grasp accuracy in corner regions.

Future work should strengthen distractor suppression via more principled noise filtering and/or richer geometric cues to improve target–distractor separability. The motion-planning stack should be extended with obstacle-aware planning to avoid degenerate configurations and to navigate around occlusions. Finally, incorporating a distortion-aware calibration model and refining low-level servo control (e.g., PID tuning) would reduce edge-of-board localization error and improve consistency between the IK-predicted pose and the physical reach across the workspace.

REFERENCES